

PROJETO FINAL

Gerenciador em Python

Fila | Pilha | Dicionário | Modularização

2026

Entrega via GitHub | Prazo: 21/05/2026

1. O que é o projeto

O projeto consiste em construir um Gerenciador em Python organizado em múltiplos arquivos, aplicando na prática as estruturas de dados estudadas em aula: lista, fila (FIFO), pilha (LIFO), dicionário e tupla. Cada grupo escolhe um dos cinco temas disponibilizados e implementa um sistema funcional e modularizado, com menu interativo e tratamento de erros.

- Data da entrega: 21/05/2026 às 23:59
- Formato de entrega: link do repositório do GitHub.
 - O repositório deve estar público
 - Deve conter o README.md
 - Deve conter os 4 arquivos.py
- Local da entrega: Ambiente virtual na pasta de PROJETO

2. Conceitos utilizados

Este projeto aplica diretamente os conceitos abaixo. O README do repositório deve conter a explicação de cada um deles com exemplos do próprio código entregue.

Fila — FIFO (First In, First Out)

Uma fila funciona como uma fila de banco: quem chega primeiro e atendido primeiro. No Python, utilizamos uma lista comum com `.append()` para inserir no final e `.pop(0)` — ou `.remove()` — para retirar do início. No projeto, a fila armazena os itens pendentes. Um novo item cadastrado entra no final da fila. O item mais antigo deve ser processado primeiro.

Exemplo de uso em código

```
fila_pendente.append(novo_item) # entra no final
fila_pendente.pop(0)            # sai pelo início (FIFO)
```

Pilha — LIFO (Last In, First Out)

Uma pilha funciona como uma pilha de pratos: o último prato colocado é o primeiro a ser retirado. No Python, usamos `.append()` para empilhar e `.pop()` — ou `reversed()` para leitura — para acessar o topo.

No projeto, a pilha armazena o histórico de itens concluídos. A tarefa mais recentemente concluída aparece primeiro na visualização do histórico.

Exemplo de uso em código

```
pilha_concluída.append(item)          # empilha no topo for item in reversed(pilha_concluída):  
# le do topo para baixo (LIFO)
```

Dicionário

Cada item do sistema é armazenado como um dicionário Python — um conjunto de pares chave-valor. Isso permite agrupar todos os atributos de um item em uma única estrutura e acessar qualquer campo pelo nome, não pela posição.

Exemplo de uso em código

```
item = { 'id': 1, 'descrição': 'Estudar listas', 'status': 'Pendente', 'prioridade': 'Alta' }
```

Lista e Tupla

A lista principal (ex: tarefas = []) armazena todos os dicionários do sistema e é mutável — itens podem ser adicionados e removidos a qualquer momento com `.append()` e `.remove()`.

As tuplas (ex: status = ('Pendente', 'Em Andamento', 'Concluído')) armazenam valores fixos que não devem ser alterados durante a execução, como categorias de status e níveis de prioridade. Por serem imutáveis, protegem o sistema contra alterações acidentais.

Modularização

O sistema é dividido em pelo menos quatro arquivos `.py`, cada um com uma responsabilidade clara:

Arquivo	Responsabilidade
dados.py	Variáveis globais: lista principal, fila, pilha, tuplas de status e prioridade
tarefas.py	Funções de regras de negócio: cadastrar, listar, atualizar status
utils.py	Funções auxiliares: exibir título, separador visual, validações
main.py	Ponto de entrada: menu principal com loop while True e desvios if/elif

3. Requisitos comuns a todos os temas

Independentemente do tema escolhido, todo projeto deve ter:

- Pelo menos 4 arquivos `.py` com responsabilidades separadas
- Lista para registro global de todos os itens
- Fila (FIFO) para itens pendentes — novos entram no final, processados saem pelo início

- Pilha (LIFO) para histórico de concluídos — exibido do mais recente ao mais antigo com `reversed()`
- Dicionário encapsulando todos os atributos de cada item
- Tupla para categorias fixas (status, prioridades)
- Menu com `while True` e opção de saída com `break`
- Tratamento de erro com `try/except` para pelo menos uma entrada do usuário
- Funções comentadas com nomes descritivos

4. Temas disponíveis

Escolha um dos cinco temas abaixo. Todos aplicam os mesmos conceitos, mas em contextos diferentes. Temas marcados como avançado exigem maior cuidado com validação de entradas e qualidade do código.

1. Biblioteca Pessoal | Intermediário

Requisitos obrigatórios

- Cadastrar livro (título, autor, ano, gênero, prioridade)
- Listar todos os livros com status atual
- Fila de livros 'a ler' — FIFO: mais antigo lido primeiro
- Pilha de livros concluídos — LIFO: último lido aparece no topo
- Atualizar status: A Ler > Lendo > Concluído
- Buscar livro por título ou autor

Funcionalidades bônus

- Filtrar livros por gênero
- Contar total de livros por status

Dica

Use a tupla `status = ('A Ler', 'Lendo', 'Concluído')` e indexe com `status[0]`, `status[1]`, `status[2]` — igual ao modelo da aula.

2. Cardápio de Restaurante | Intermediário

Requisitos obrigatórios

- Cadastrar pedido (mesa, item, quantidade, prioridade)
- Listar pedidos em aberto por mesa
- Fila de preparo — FIFO: pedido mais antigo preparado primeiro
- Pilha de pedidos entregues — LIFO: último entregue aparece no topo
- Atualizar status: Aguardando > Em Preparo > Entregue
- Cancelar pedido (remover da fila de preparo com `.remove()`)

Funcionalidades bônus

- Calcular total da conta por mesa
- Exibir itens mais pedidos

Dica

O cancelamento usa `fila_preparo.remove(pedido)`. Verifique antes se o pedido está na fila para evitar erro.

3. Academia / Treinos | Avançado

Requisitos obrigatórios

- Cadastrar aluno (nome, modalidade, dias/semana, objetivo)
- Listar alunos ativos com status
- Fila de atendimento presencial — FIFO

Funcionalidades bônus

- Contar total de sessões por aluno
- Listar alunos por modalidade

Dica

- Pilha de sessões concluídas por aluno — LIFO
- Atualizar status: Ativo > Suspenso > Cancelado
- Registrar check-in de treino com data

Guarde a pilha de sessões dentro do próprio dicionário do aluno: `aluno['sessões'] = []`. Use `.append()` para empilhar e `reversed()` para exibir.

4. Controle Financeiro Pessoal | Avançado

Requisitos obrigatórios

- Cadastrar transação (descrição, valor, tipo: receita/despesa, categoria)
- Listar todas as transações com status
- Fila de despesas pendentes — FIFO: mais antiga paga primeiro
- Pilha de pagamentos realizados — LIFO: último pagamento no topo
- Atualizar status: Pendente > Pago
- Exibir saldo atual (receitas - despesas pagas)

Funcionalidades bônus

- Filtrar transações por categoria
- Total gasto por categoria

Dica

Use `try/except` ao ler o valor: `valor = float(input("Valor: "))` dentro de `try`. Sem isso, texto no lugar do número quebrará o programa.

5. Locadora de Filmes | Básico

Requisitos obrigatórios

- Cadastrar filme (título, gênero, ano, disponível: sim/não)
- Listar catálogo com status de disponibilidade
- Fila de reservas — FIFO: primeiro a reservar aluga primeiro
- Pilha de devoluções — LIFO: última devolução aparece primeiro
- Registrar aluguel (marca filme como indisponível)
- Registrar devolução (marca filme como disponível)

Funcionalidades bônus

- Buscar filme por título
- Listar filmes disponíveis por gênero

Dica

Tema ideal para consolidar os conceitos. Foque em deixar a lógica de FIFO e LIFO muito explícita no código e nos comentários.

5. Entrega via GitHub

O projeto deve ser entregue como um repositório público no GitHub. O link do repositório deve ser submetido na plataforma da disciplina até o prazo divulgado.

Estrutura esperada do repositório

Organização dos arquivos

```
gerenciador_[tema]/ | |-- main.py |-- dados.py |-- tarefas.py |-- utils.py |-- README.md
```

O que o README deve conter

O arquivo README.md é obrigatório e faz parte da avaliação. Ele deve ser escrito em Markdown e conter as seguintes seções:

1. Título e descrição do projeto

Nome do projeto, tema escolhido, integrantes e período/turma.

2. Explicação conceitual

Explique com suas próprias palavras — usando exemplos do seu próprio código — o que e como funciona cada estrutura de dados usada no projeto:

- O que é uma Fila (FIFO) e onde ela aparece no seu sistema
- O que é uma Pilha (LIFO) e onde ela aparece no seu sistema
- Como o Dicionário foi usado para representar cada item
- Qual a diferença entre Lista e Tupla e por que cada uma foi usada
- O que é modularização e como o projeto foi dividido em arquivos

3. Como executar o projeto

Instruções claras para rodar o programa localmente, por exemplo:

- Versão do Python necessária (Python 3.10+)
- Comando para executar: `python main.py`
- Não são necessárias bibliotecas externas

4. Funcionalidades implementadas

Lista das funcionalidades entregues, incluindo quais bônus foram implementados (se houver).

5. Dificuldades e aprendizados

Breve relato (mínimo 5 linhas) descrevendo o que foi mais difícil de implementar e o que o grupo aprendeu ao longo do desenvolvimento.

Dica para o README

Use blocos de código Markdown (```) para mostrar trechos do seu código ao explicar os conceitos. Um README bem escrito demonstra que o grupo entendeu o que implementou — não apenas que o código funciona.

6. Regras gerais

- O arquivo main.py deve ser o ponto de entrada: executar python main.py deve iniciar o programa sem erros.
- O código deve funcionar no Python 3.10 ou superior sem bibliotecas externas.
- O repositório deve ser público e o link deve estar acessível no prazo de entrega.
- Projetos com código idêntico ou muito similar a outros grupos recebem nota zero para ambos.
- O uso de IA para gerar o código é proibido. O grupo deve ser capaz de explicar qualquer trecho durante a produção do projeto em sala de aula.

Atrasos

Entregas após o prazo perdem 1,0 ponto por dia, até o limite de 2 dias corridos. Após esse período a entrega não será aceita.